From Neural Networks to Scientific Machine Learning for Practical Physical Problems

Noah Holicky

November 8, 2026

Introduction

Purpose of this document

To learn deep learning with the focus of looking towards physics informed neural networks. Instead of spending hours focusing on building all the foundations with textbooks and extensive study, I am trying an unconventional approach by using generative AI to tailor a course to me. I have a background in pure mathematics and theoretical physics and so am approaching this from a heavily mathematical foundation.

Goals

Contents

Introduction i

Notation iv

I Neural Network Foundations 1

1 Neural Networks as Parameterised Functions 2
1.1 Parameterised models, feedforward neural networks and nonlinearity 2

2 Training and Optimisation 4
2.1 Input Sensitivity and Parameter Sensitivity 4
2.2 Loss functions: what training minimises 5
2.3 Gradient-Based Optimisation 7
2.4 Curvature and Learning Rates 9

3 Automatic Differentiation and Backpropagation 11
3.1 Automatic Differentiation 11
3.2 Backpropagation 15

4 Small Projects: Linear Regression and a Multilayer Network 19

II Practical Deep Learning 20

5 Project: Function and Derivative Approximation 21

III Physics-Informed Neural Networks 22

6 Differential Equations as Learning Problems 23
6.1 Governing equations, residuals and loss functionals 23
6.2 Residual-Error Relations and Stability Estimates 26
6.3 From empirical loss to solution accuracy 27

7 Project: An ODE PINN 28
7.1 Goals 28
7.2 Theory and Derivations 28
7.3 Exercises and Implementation 28
7.4 Results and Reflection 28
7.5 References 28

8 Project: The Heat Equation **29**
8.1 Goals . . . 29
8.2 Theory and Derivations . . . 29
8.3 Exercises and Implementation . . . 29
8.4 Results and Reflection . . . 29
8.5 References . . . 29

9 Inverse Problems **30**
9.1 Goals . . . 30
9.2 Theory and Derivations . . . 30
9.3 Exercises and Implementation . . . 30
9.4 Results and Reflection . . . 30
9.5 References . . . 30

Notation

Symbol	Meaning
$\mathbb{R}$	Real numbers
$x \in \mathbb{R}^n$	Input vector
$\theta \in \mathbb{R}^p$	Model parameters
f_θ	Parameterised model
$\mathcal{L}(\theta)$	Loss function
$\nabla_\theta \mathcal{L}$	Gradient of the loss with respect to parameters

Part I

Neural Network Foundations

Chapter 1

Neural Networks as Parameterised Functions

1.1 Parameterised models, feedforward neural networks and nonlinearity

Arguably the most motivating factor in the development of neural networks is the idea of trying to most accurately describe an unknown function using finitely many unknown numbers. This is best done through a parameterised model.

Definition 1.1 (Parameterised model)**.** Let Θ, X, and Y be the parameter, input, and output spaces, respectively. A **parameterised model** is a map

$$F : \Theta \times X \to Y.$$

with the property that for each fixed $\theta \in \Theta$, we have an associated parameterised function:

$$f_\theta : X \to Y, \qquad f_\theta(x) = F(\theta, x)$$

The parameterised model, F, determines the family of functions $\{f_\theta\}_{\theta \in \Theta}$.

Remark 1.2. When we talk about *learning*, we mean selecting functions from a parameterised family, according to an objective. For instance, minimisation. Asking the questions of which functions can a family represent and how should we choose among them lead to the idea of a models' expressive power (i.e. ability to represent a range of functions/problems) and secondly, training.

Remark 1.3. As a quick reminder, an **affine function** is a function consisting of a linear transformation and then a translation. A basic example if $f(x) = ax + b$, for $a, b \in \mathbb{R}$.

Definition 1.4 (Feedforward neural network)**.** A standard **feedforward neural network** is a parameterised model constructed by the composition of affine maps and activation functions:

$$h_0 = x, \qquad h_\ell = \sigma_\ell(W_\ell h_{\ell-1} + b_\ell), \quad \ell = 1, \ldots, L-1,$$

with output

$$f_\theta(x) = W_L h_{L-1} + b_L.$$

Note that the activation functions, σ_ℓ, act componentwise. The parameter vector θ collects all the adjustable weights and biases used in training the model. Moreover we call the vectors h_ℓ **hidden representations**.

Remark 1.5. The hidden representations are simply the intermediate quantities computed from the input which are not seen from the output. They are 'hidden' in the sense that they are never seen without explicitly computing each composition.

Example 1.6. An example of a one hidden layer feedforward neural network is defined by functions $f_\theta : \mathbb{R}^n \times \mathbb{R}^q$ of the form:

$$f_\theta(x) = W_2 \tanh(W_1 x + b_1) + b_2$$

where $W_1 \in \mathbb{R}^{m\times n}, b_1 \in \mathbb{R}^m, W_2 \in \mathbb{R}^{q\times m}$ and $b_2 \in \mathbb{R}^q$.

Definition 1.7 (Architecture)**.** The **architecture** of a neural network specifies the network construction. This includes the dimensions of the layers, connectivity, and activation functions.

Remark 1.8. Fixing an architecture gives a parameterised family, $F : \Theta \times X \to Y$; fixing its parameters, $\theta \in \Theta$ gives a particular function f_θ.

Remark 1.9. A particularly important point parameterization is not unique. Namely, choosing different parameter values does not guarantee a different function.

Proposition 1.10 (Affine collapse)**.** *Any finite composition of affine maps is affine.*

Proof. A very easy induction argument will show this is true. □

As a result of 1.10, a network containing only affine operations will have an affine input-output map. The value of neural networks now becomes particularly important when we make the activation functions non-linear, namely, the overall input-output function becomes non-linear.

Remark 1.11 (Two meanings of nonlinearity). Nonlinearity in the input is distinct from nonlinearity in the parameters. For example,

$$f_{(a,b,c)}(x) = ax^2 + bx + c$$

is linear in (a, b, c) but generally nonlinear in x. On the other hand though:

$$f_{(a,b,c)}(x) = a^2 x + bx + c$$

is nonlinear in (a, b, c), but linear in x.

Remark 1.12 (Scientific representations). A PINN can represent a candidate solution $u_\theta(t, x)$. A neural ODE represents a vector field $v_\theta(t, z)$; its trajectory is obtained by solving the resulting ODE. Importantly, this raises the point that we need to choose our model according to whether we want to represent a solution, or whether we want to represent its governing dynamics.

Chapter 2

Training and Optimisation

2.1 Input Sensitivity and Parameter Sensitivity

There are two types of sensitivity when it comes to a neural network. Firstly, we have **input sensitivity** which refers to how the output of the model changes when the input x changes with fixed parameters θ. Secondly, fixing the input x, the **parameter sensitivity** is how the output changes when the parameters θ change.

Naturally, we can utilise the field of calculus to study these sensitivities precisely, at least when we have differentiable models like those over Euclidean spaces or in certain contexts, over differentiable manifolds. Indeed, fixing a point $(\theta, x) \in \Theta \times X$, we can look at the partial differentials of the parameterised model.

Definition 2.1 (Input and parameter differentials and Jacobians)**.** Let $(\theta, x) \in \Theta \times X$ be a fixed point. The input and parameter partial differentials are, respectively, the following linear maps:

$$D_x F(\theta, x) : \mathbb{R}^n \to \mathbb{R}^q$$

$$D_\theta F(\theta, x) : \mathbb{R}^p \to \mathbb{R}^q$$

over Euclidean space. One can readily extend this in the usual way to get the following **input Jacobian** and **parameter Jacobian**, $J_x F \in \mathbb{R}^{q \times n}$ and $J_\theta F \in \mathbb{R}^{q \times p}$, respectively.

Remark 2.2. In multivariable calculus, the Jacobian matrix is the object that encodes all of the first-order partial differential information in standard coordinates. More generally, for a smooth map between smooth manifolds, the differential at each point is a linear map between the corresponding tangent spaces. In local coordinates, this linear map is represented by a Jacobian matrix.

From variational calculus, the differentiability of the models mean that to a first-order approximation:

$$F(\theta + \delta\theta, x + \delta x) = F(\theta, x) + J_\theta F \delta\theta + J_x F \delta x + o(||(\delta\theta, \delta x)||) \tag{2.1}$$

for some small variation $(\delta\theta, \delta x)$. This allows us to see a models **local linear response** by neglecting higher order terms. This is very useful in the training phase. The first-order terms $J_x F \delta x, J_\theta F \delta\theta$ represent first order changes and so are a good way of quickly looking at changes. For exactness, we need to look at $F(\theta, x + \delta x) - F(\theta, x)$ and $F(\theta + \delta\theta, x) - F(\theta, x)$, respectively.

Remark 2.3. It is important to note that models can have curvature (or non-zero gradient) with respect to one of the input or parameters, whilst having no curvature with respect to the other. This means specifying the variable being differentiated is particularly important.

2.2 Loss functions: what training minimises

The whole aim of building a neural network, is to build a model which most aptly describes a physical problem. In order to quantify this, one has to firstly have some quanitifiable way of describing the model's 'closeness' to a true value. This is where a loss function comes into play.

To cement this more concretely, we mathematically introduce observations as the collection of tuples of (independent variables, dependent variables):

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$$

The goal of training now becomes to find the parameterised function f_θ such that $f_\theta(x_i)$ best agrees with the actually measured/observed outputs, y_i. Loss functions, quantitatively describe this 'closeness'. As such, training a model now becomes an optimisation problem.

Definition 2.4 (Sample Loss)**.** A **sample loss** is a function:

$$\ell : Y \times Y \to [0, \infty)$$

that assigns a penalty to a prediction $\hat{y}$ and a target y, written as $\ell(\hat{y}, y)$.

Definition 2.5 (Empirical risk)**.** We define the **empirical risk** (also known as the average training loss) as the average sample loss over all observation points:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \ell(f_\theta(x_i), y_i) \tag{2.2}$$

Training then becomes about minimising this over the parameter space, namely, if one exists, finding θ^* with the property:

$$\theta^* \in \arg\min_{\theta \in \Theta} \mathcal{L}(\theta)$$

Definition 2.6 (Residual)**.** For the ith observation, the ith **residual** is defined as:

$$r_i(\theta) = f_\theta(x_i) - y_i$$

In a normed vector space, like Euclidean space, we can define our sample loss using a metric $||\cdot||_N$ (N is the norm in the space). Using this to define our sample loss, we can write the empirical risk in the convenient form:

$$\mathcal{L}(\theta) = \frac{1}{2N} \sum_{i=1}^{N} ||r_i(\theta)||_2^2 \tag{2.3}$$

where the $1/2$ is an arbitrary choice for convenience when differentiating.

Remark 2.7. Choosing different norms enables us to choose different ways of weighting the components.

Remark 2.8. In the case that $\mathcal{L}(\theta) = 0 \iff f_\theta(x_i) = y_i$ for all training observation. Importantly, this only says the model is accurate on the data set, not elsewhere. It doesn't mean that on a different data set it will achieve the same results.

Assuming that $f_\theta(x_i)$ is differentiable with respect to θ, we can write the ith Jacobian as:

$$J_i = D_\theta f_\theta(x_i) \in \mathbb{R}^{q \times p}$$

at least over Euclidean space. For a small perturbation in the parameter $\delta\theta$, applying the chain rule to the empirical risk yields:

$$D_\theta \mathcal{L}[\delta\theta] = \frac{1}{N}\sum_{i=1}^{N} r_i^T J_i \delta\theta$$

Recalling that $D_\theta F[\delta\theta] = \nabla_\theta F \cdot \delta\theta$ yields the following convenient form:

$$\nabla_\theta \mathcal{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N} J_i^T r_i \tag{2.4}$$

where we have used the fact that the inner product can be written as the product of matrices $\nabla_\theta \mathcal{L} \cdot \delta\theta = (\nabla_\theta \mathcal{L})^T \delta\theta$.

Example 2.9. In the case that the output is a scalar i.e. $f : \mathbb{R}^n \to \mathbb{R}$ we recall that the Jacobian is simply a row matrix of the form:

$$J_f(x) = \begin{pmatrix} \frac{\partial f}{\partial x_1} \cdot \frac{\partial f}{\partial x_n} \end{pmatrix}$$

Conveniently, this is just the transpose $\nabla f(x)^T$. Consequently, this means that the Jacobian terms J_i^T are of the form:

$$J_i^T = \left(\nabla_\theta f_\theta(x_i)^T\right)^T = \nabla_\theta f_\theta(x_i)$$

And from 2.6, $r_i(\theta) = f_\theta(x_i) - y_i$ so that 2.4 becomes:

$$\nabla_\theta \mathcal{L} = \frac{1}{N}\sum_{i=1}^{N} (f_\theta(x_i) - y_i)\nabla_\theta f_\theta(x_i)$$

Remark 2.10. The above example is subtly important as it illustrates an important point. Namely, it makes evident that the parameter derivative measures how changes to the parameters affect the prediction of the model. Since the residual measures the current prediction error, the two together measure how changing the parameters affects the objective.

Another subtle point is as follows. If we consider a single scalar observation the empirical risk simplifies as:

$$\mathcal{L}(\theta) = \frac{1}{2}(f_\theta(x) - y)^2$$

so that taking the gradient with respect to the parameters readily yields:

$$\nabla_\theta \mathcal{L} = (f_\theta(x) - y)\nabla_\theta f_\theta(x)$$

If the prediction is exact then $\nabla_\theta \mathcal{L} = 0$ even if $\nabla_\theta f_\theta(x) \neq 0$. Consequently since training aims to minimise the loss, the most important gradient is $\nabla_\theta \mathcal{L}$ rather than the gradient of the prediction alone $\nabla_\theta f_\theta(x)$ since this is what we are trying to minimise.

2.3 Gradient-Based Optimisation

The aim of this section is to optimise the parameters in order to minimise the loss function. Anyone who has studied basic calculus understands that this amounts to trying to find a local/global minimum (depending on your domain/data set). But how do we do this in an iterative manner for when we program a computer. The natural way of doing so is by adjusting the parameters in the direction that reduces the loss. For this we need a descent direction.

Definition 2.11 (Descent direction)**.** A vector v is called a **descent direction** at the parameter θ if:

$$\nabla\mathcal{L}(\theta)^T v < 0$$

In other words, the loss gradient decreases in the direction of v.

Remark 2.12. Note it is important to realise we could have equally written this equation in the more intuitive form:

$$\nabla\mathcal{L}(\theta) \cdot v < 0$$

In which case we recognise this as the dot product between the vectors. In other words, the loss gradient is negative in the direction of v, or more intuitively, the loss decreases in the direction of v.

Naturally, this tells us which direction to move in order to reduce the loss. More precisely, one can show using real analysis that this guarantees that $\mathcal{L}(\theta + tv) < \mathcal{L}(\theta)$ for sufficiently small $t > 0$.

Intuitively, it is easy to realise that the direction of steepest descent is that directly opposite to the loss gradient. But one can also see this mathematically. Let us assume Euclidean coordinates and metric and that the gradient of the loss is nonzero, $\nabla\mathcal{L}(\theta) \neq 0$. Arbitrarily we will let $g = \nabla\mathcal{L}(\theta)$. The goal is to then find a unit vector v (unit because it is a normalised vector) which is precisely in this direction of steepest descent i.e. $||v||_2 = 1$. The Cauchy-Schwarz theorem tells us that:

$$|g^T v| \leq ||g||_2 \implies g^T v \geq -||g||_2$$

Now equality occurs when the vectors are linearly dependent, i.e. in the exact same direction. Letting $v = cg$ and recalling that $||v||_2 = 1$ we must have that:

$$|c|||g||_2 = 1 \implies c = \pm\frac{1}{||g||_2}$$

So if $v = cg$ then choosing:

$$v = -\frac{g}{||g||_2}$$

Guarantees that this achieves the negative lower bound i.e. this vector minimises the first-order loss change. We can easily see this:

$$g^T v = -\frac{g^T g}{||g||_2} = -\frac{||g||_2^2}{||g||_2} = -||g||_2$$

which is precisely the Cauchy-Schwarz lower bound.

Remark 2.13. All the above discussion is doing is mathematically validating our intuitive argument, proving that the descent direction vector that minimises the loss is indeed in the negative direction of the gradient of the loss. The factor of $\frac{1}{||\nabla\mathcal{L}(\theta)||_2}$ is purely for normalization.

So equipped with this, how do we actually apply it? Through an iterative algorithm known as gradient descent.

Definition 2.14 (Gradient descent)**. Gradient descent** is the iterative algorithm that updates a models parameters according to the rule:

$$\theta_{k+1} = \theta_k - \eta_k \nabla \mathcal{L}(\theta_k) \tag{2.5}$$

where $\eta_k > 0$ is known as the **learning rate**.

If we now consider a small change to the parameters, the variational approach that we took in 2.1 yields:

$$\mathcal{L}(\theta + \delta\theta) = \mathcal{L}(\theta) + \nabla \mathcal{L}(\theta)^T \delta\theta + o(||\delta\theta||)$$

What is really nice is that if we employ the method of gradient descent, our change in parameter between iterations is simply $\delta\theta = -\eta_k \nabla \mathcal{L}(\theta_k)$. Plugging this into the above, we can readily see that the first-order (linear) change is just:

$$\Delta \mathcal{L}_{linear} = \nabla \mathcal{L}(\theta)^T \delta\theta$$

$$= -\nabla \mathcal{L}(\theta_k)^T \eta_k \nabla \mathcal{L}(\theta_k)$$

which by the Euclidean metric yields:

$$\Delta \mathcal{L}_{linear} = -\eta_k ||\nabla \mathcal{L}(\theta_k)||_2^2 \tag{2.6}$$

Remark 2.15. When $\nabla \mathcal{L}(\theta_k) \neq 0$ this is negative and acts to decrease the loss when the learning rate is sufficiently small (if it is too big it can go past the minimum). Importantly, this is only a prediction based on the local linear approximation and it doesn't take into account higher order changes.

Suppose we reach a point in which the gradient is zero. This is the usual stationary point.

Definition 2.16 (Stationary point)**.** A point θ is **stationary** if

$$\nabla \mathcal{L}(\theta) = 0$$

At such a point, gradient descent does not make any update. Very importantly though, as in regular calculus, a stationary point does not guarantee the point is a minimum.

Example 2.17. The following is a very important example that demonstrates why choosing an appropriate learning rate is important. Consider the affine model given by:

$$f_{(a,b)}(x) = ax + b$$

with observations $(x_1, y_1) = (0, 1)$ and $(x_2, y_2) = (1, 3)$. A simple exercise leads to the following loss function:

$$\mathcal{L}(a, b) = \frac{1}{4}[(b-1)^2 + (a+b-3)^2]$$

where we have selected the sample loss as $\ell(\theta) = ||r_i(\theta)||_2^2$.

Now in the case that $(a, b) = 1$ we see that $\mathcal{L} = 5/4$ and $\nabla \mathcal{L} = \begin{pmatrix} -1 \\ -3/2 \end{pmatrix}$. Choosing a learning rate of $\eta = 0.2$, the updated parameters easily fall out as:

$$\theta_{k+1} - \theta_k - \eta_k \nabla \mathcal{L}(\theta_k) = \begin{pmatrix} 1 \\ 0 \end{pmatrix} - 0.2 \begin{pmatrix} -1 \\ -3/2 \end{pmatrix}$$

which gives updated parameters $(1.2, 0.3)$. Calculating the new loss we see that $\mathcal{L}_{new} = 0.685$. Now suppose we choose a different learning rate of $\eta = 2$. Recalculating we get updated parameters of $\theta_{new'} = (3, 3)$. Moreover the loss comes out as $\mathcal{L}_{new'} = 13/4$. Clearly, $\mathcal{L}_{new'} >> \mathcal{L}_{new}$ even though we used gradient descent and updated our parameters in the descent direction.

So why did this occur? Well, if we look back to our motivation for choosing gradient descent, we recall that the loss decreases $\mathcal{L}(\theta + tv) < \mathcal{L}(\theta)$ when v is a descent direction and $t > 0$ is *sufficiently small*. In our example $0.2 < 2$ and although not remarkably smaller in the grand scheme of the real numbers, it is considerably smaller for an iterative algorithm. This is a particularly important point. That gradient descent is not just about choosing the descent direction, but about choosing the appropriate learning rate. This is just a basic ϵ argument. Namely, that there exists $\epsilon > 0$ such that:

$$0 < t < \epsilon \implies \mathcal{L}(\theta + tv) < \mathcal{L}(\theta)$$

But in our example we chose a $\eta > \epsilon$. Intuitively, this is like following a gradient descent vector 'past' a minimum (supposing that it occurs on that line).

2.4 Curvature and Learning Rates

As is known from calculus, understanding the behaviour of a function does not always come exclusively from first-order behaviour. We need to turn to higher-order contributions to understand the full behaviour of a function. As usual in multivariable calculus we do this through the Hessian.

Definition 2.18 (Hessian)**.** Let $\mathcal{L} : \mathbb{R}^p \to \mathbb{R}$ be a twice continuously differentiable function. The Hessian is the symmetric $p \times p$ matrix given by:

$$H(\theta) = \nabla^2 \mathcal{L}(\theta), \qquad H_{ij}(\theta) = \frac{\partial^2 \mathcal{L}}{\partial \theta_i \partial \theta_j}$$

A basic second order Taylor expansion yields the following expression:

$$\mathcal{L}(\theta + \delta\theta) = \mathcal{L}(\theta) + g^T \delta\theta + \frac{1}{2}\delta\theta^T H(\theta)\delta\theta + o(||\delta\theta||^2) \tag{2.7}$$

where again $g = \nabla\mathcal{L}(\theta)$.

Let us consider now what happens to 2.7 if we move in a descent direction. Let $\delta\theta = -\eta g$, then:

$$\begin{aligned}\mathcal{L}(\theta - \eta g) &= \mathcal{L}(\theta) - \eta g g^T + \frac{1}{2}\eta g^T H(\theta)\eta g + o(||\eta g||^2) \\ &= -\eta||g||^2 + \frac{\eta^2}{2} g^T H(\theta) g + o(\eta^2)\end{aligned}$$

since $||\eta g||^2 = \eta^2 ||g||^2 = \eta^2$ since $||g^2|| = 1$. Rearranging, we find:

$$\mathcal{L}(\theta - \eta g) - \mathcal{L}(\theta) = -\eta||g||^2 + \frac{\eta^2}{2} g^T H(\theta) g + o(\eta^2) \tag{2.8}$$

This explains precisely our observation Example 2.17. Indeed, the first term is linear in η, whilst the second is quadratic in η. As such, the quadratic term will dominate when η is not sufficiently small, as we saw previously.

Because the Hessian determines second-order effects, we can use it as a way of bounding our learning-rate so as to avoid the implications of Example 2.17. Let $||\cdot||_{op}$ be the matrix norm that is induced by the Euclidean norm. Suppose that the Hessian for our loss function satisfies the following inequality:

$$||H(z)||_{op} \leq \beta$$

throughout the given step segment $(k \to k+1)$ and where $\beta > 0$. Applying Taylor's theorem with an integral remainder yields the following expression:

$$\mathcal{L}(\theta - \eta g) \leq \mathcal{L}(\theta) - \eta \left(1 - \frac{\beta \eta}{2}\right) ||g||^2$$

In the case $g \neq 0$ it is not hard to see that $\mathcal{L}(\theta - \eta g) < \mathcal{L}(\theta)$ (i.e. a strict decrease) when:

$$0 < \eta < \frac{2}{\beta} \leq \frac{2}{||H(\theta)||_{op}}$$

Remark 2.19. One can see that if curvature is high (i.e. the norm of the Hessian is large), the bound gets smaller and so we have to decrease our learning rate (or smaller updates to our parameters).

Chapter 3

Automatic Differentiation and Backpropagation

3.1 Automatic Differentiation

Because of the scale of a neural network, it becomes incredibly impractical to differentiate every parameter and moreover, entire neural networks. However, the method of automatic differentiation rectifies this issue.

To put this more explicitly, let $f_\theta : \mathbb{R}^n \to \mathbb{R}^q$ with $\theta \in \mathbb{R}^p$ be a neural network. Let (x, y) be an observation and consider a basic loss function:

$$\mathcal{L}(\theta) = \frac{1}{2}||f_\theta(x) - y||^2$$

In order to train the model we must now information about the gradient of the loss with respect to the parameters $\nabla_\theta \mathcal{L} \in \mathbb{R}^p$. Previously, we found a formula for this:

$$\nabla_\theta \mathcal{L} = J_\theta f_\theta(x)^T (f_\theta(x) - y)$$

This is great for basic neural networks. But what happens when the number of layers and/or parameters is on the order of thousands? Millions even? It becomes exceptionally complex. This is where AD comes into full effectiveness, exploiting the way the network is construction to reuse/repeat calculations rapidly.

Definition 3.1 (Automatic differentiation)**. Automatic differentiation** is the process of evaluating the derivatives of a computation by applying the chain rule to the constituent operations.

Remark 3.2. AD does not need to construct the full Jacobian matrices (i.e. a closed-form expression for the derivative of the overall neural network) but rather, derivatives of the operations used to construct the network. For instance, consider the function $F(\theta) = g(f(\theta))$. Applying the chain rule yields:

$$J_F(\theta) = J_g(f(\theta)) J_f(\theta)$$

Simply knowing J_g and J_f is enough to understand J_F.

Rather than performing symbolic differentiation by constructing an expression for the total derivative or relying on perturbations of the parameters via variational analysis, AD propagates derivative information through an entire computation (neural network). In other words, it evaluates derivatives using local derivative rules and intermediate numerical values.

Definition 3.3 (Computational graph)**.** A **computational graph** is an acyclic graph whose nodes represent inputs or intermediate quantities, and whose edges represent dependencies. It describes how a calculation depends on the inputs and intermediate quantities.

Remark 3.4. Let us digest this terminology. Simply, a computational graph is a way of recording the operations in a calculation and the dependencies between the inputs and outputs.

Example 3.5. For the function $f_\theta(x) = a\tanh(bx + c) + d$, with squared-error loss, we can break this down into the simplified number of steps:

1. $z = bx + c$
2. $h = \tanh z$
3. $\hat{y} = ah + d$
4. $r = \hat{y} - y$
5. $\mathcal{L} = \frac{1}{2}r^2$

A computational graph tells us that z depends on b, x, c, h depends on z, $\hat{y}$ depends on a, h, d and the loss depends on $\hat{y}$ through the residual. AD then works by composing the derivatives of these operations via the chain rule since each of these equations (1-5) are very simple operations with known derivatives.

Definition 3.6 (Forward pass)**.** A **forward pass** is the evaluation of a function and propagation of its derivatives in dependency order. It starts from the inputs and works its way through, producing the intermediate quantities and final loss.

Remark 3.7. Put differently, a forward pass starts at the input and works its way *forwards* through the computational graph.

Remark 3.8. Again we recall the use of this. For many neural networks with multiple layers, the entire network becomes incredibly complicated if viewed as one equation. Conversely, broken down into steps as in 3.5 and pieced together using the chain rule, it becomes much more manageable and easier to compute.

Suppose that we have a basic function defined by two steps:

$$u = f(\theta), \qquad v = g(u)$$

Then for some perturbation/variation to the input which is commonly written $\dot{\theta}$ (denotes a directional derivative, not a time derivative), it's not hard to see that the chain rule yields:

$$\dot{v} = J_f(\theta)\dot{\theta}, \qquad \dot{v} = J_g(u)\dot{u} = J_g(u)J_f(\theta)\dot{\theta}$$

Applying the chain rule in this direction is known as forward-mode, since we are applying it from the input θ, forward. Naturally, this yields:

Definition 3.9 (Forward-mode AD)**. Forward-mode automatic differentiation** propagates directional derivatives through a computational graph in the same order as the forward evaluation.

Consequently, for any function $F : \mathbb{R}^p \to \mathbb{R}^q$, a forward pass combined with any vector $v \in \mathbb{R}^p$ computes the *Jacobian-vector product*:

$$J_F(\theta)v$$

This is simply the product of the Jacobian for that function, with a vector.

Remark 3.10. Take the simple function $u = ab$. Differentiating with the product rule yields, $\dot{u} = b\dot{a} + a\dot{b}$. If we choose $(\dot{a}, \dot{b}) = (1, 0)$ we are choosing to look at the change in the direction of a, namely, the sensitivity of the neural network to a. Conversely, $(0, 1)$ measures the sensitivity to b.

In fact we can be more general here. For a scalar loss, let $s = e_j$ be the jth coordinate vector. The JVP then yields:

$$J_{\mathcal{L}} e_j = \frac{\partial \mathcal{L}}{\partial \theta_j}$$

Or in other words, the gradient in the jth direction.

It is important to realise here that forward-mode AD very quickly becomes incredibly expensive. When the number of parameters p in a model increases, so does the number of directional derivatives and therefore the complexity of the resultant forward-mode AD.

Remark 3.11. Before we move on it is important to note that forward-mode differentiation propagates directional derivatives since the JVP selects the jth coordinate. Composition, or, forward-mode differentiation, does not change this.

If forward-mode AD acts to address the question of how specified input perturbations affect later quantities, reverse mode AD does the opposite. Namely it asks; how does each intermediate quantity affect a specified final objective?

Definition 3.12 (Intermediate loss sensitivity)**.** For some scalar intermediate quantity u, the **intermediate loss sensitivity**, or more commonly called an **adjoint**, is the gradient of the loss function with respect to the intermediate quantity. It is denoted by $\bar{u}$ and defined:

$$\bar{u} := \frac{\partial \mathcal{L}}{\partial u} \tag{3.1}$$

If u is a vector quantity then we simply have:

$$\bar{u} = \nabla_u \mathcal{L}$$

Almost always we will have more than one intermediate quantity. Take for instance the simple function above $f_\theta(x) = a \tanh(bx + c) + d$ which had 5 intermediate quantities. For a simple case, let v be the next intermediate quantity defined by $v = \phi(u)$ with $u \in \mathbb{R}^n, v \in \mathbb{R}^m$. If we assume the loss depends on u through v, for a small perturbation of u, δu,

$$\delta v \approx J_\phi(u)\delta u$$

with first-order loss change being:

$$\delta \mathcal{L} \approx \bar{v}^T \delta v = \bar{v}^T J_\phi(u)\delta u$$

$$= \left(J_\phi(u)^T \bar{u}\right)^T \delta u$$

Or in other words:

$$\bar{u} = J_\phi(u)^T \bar{v} \tag{3.2}$$

Continuously applying this backwards determine the derivatives;

Definition 3.13 (Reverse-mode AD)**. Reverse-mode automatic differentiation** propagates output sensitivities backwards through the computational graph by using transpose-Jacobian vector products.

Remark 3.14. Almost always, scalar valued losses are used. Naturally for scalar losses, the loss does not depend on itself, and so when building the neural network, we always initialise the loss adjoint to 1, $\bar{\mathcal{L}} = 1$.

It is trivial, but very enlightening, to see that reverse-mode AD does in deep work backwards in determining the gradient. Recalling the fact we are composing functions, the total Jacobian is simply:

$$J_F = J_L J_{L-1} ... J_1$$

Which taking the transpose yields:

$$J_F^T = (J_L J_{L-1} ... J_1)^T = J_1^T ... J_{L-1}^T J_L^T$$

And since these are matrices the rightmost factor acts first. In other words, applied to an output sensitivity we start from the final node and work backwards.

It is often the case however that one runs into multiple branches leading into the same intermediate quantity. Namely, in most neural networks a neuron's output feeds into many neurons in the next layer. The overall loss sensitivity to that neuron must take into account all of these effects. The accumulation rule does this and says that the backward sensitivity is the sum of all these losses.

Example 3.15. Take for instance a function with steps:

$$u = ab, \quad v = a^2, \quad s = u + v, \quad \mathcal{L} = \frac{1}{2}(s - y)^2$$

Suppose we want to know what $\bar{u} = \frac{\partial \mathcal{L}}{\partial a}$ is. Arbitrarily letting $r = s - y$ we can easily see that:

$$\bar{s} = \frac{\partial \mathcal{L}}{\partial s} = r, \quad \bar{u} = r, \quad \bar{v} = r$$

In the next steps:

$$\frac{\partial s}{\partial u} = 1, \quad \frac{\partial s}{\partial v} = 1$$
$$\frac{\partial u}{\partial a} = b, \quad \frac{\partial v}{\partial a} = 2a$$

Thus, putting these altogether we write and noting that $\frac{\partial y}{\partial a} = 0$ since y is an observation variable:

$$\frac{\partial \mathcal{L}}{\partial a} = \frac{\partial \mathcal{L}}{\partial u}\frac{\partial u}{\partial a} + \frac{\partial \mathcal{L}}{\partial v}\frac{\partial v}{\partial a}$$
$$= \bar{u}\frac{\partial u}{\partial a} + \bar{v}\frac{\partial v}{\partial a} = rb + 2ra$$

Therefore, $\bar{a} = rb + 2ra$. This example, clearly illustrates that both branches contribute to the sensitivity.

Proposition 3.16 (Accumulation rule)**.** *When a variable influences the loss through several directly dependent quantities, it's backwards sensitivity is the sum of the contributions through those quantities.*

For example, if the nodes of the neural network are scalars;

$$\bar{u} = \sum_{v \text{ directly depending on } u} \bar{v}\frac{\partial v}{\partial u} \tag{3.3}$$

3.2 Backpropagation

Definition 3.17 (Backpropagation)**. Backpropagation** is reverse-mode automatic differentiation applied to a neural-network computation.

Remark 3.18. Note that this arbitrary difference 'applied to a neural-network computation' distinguishes backpropagation in the context of computer science as opposed to the more general mathematical technique - reverse-mode AD.

Example 3.19. Let us return to the example we have used throughout, the hyperbolic tangent network 3.5. For the sake of convenience, we will rewrite the equations defining the network:

$$z = bx + c, \quad h = \tanh z, \quad \hat{y} = ah + d, \quad r = \hat{y} - y, \quad \mathcal{L} = \frac{1}{2}r^2$$

Now it's not hard to see that with (x, y) as our training data, a, b, c, d are our parameters. Namely, $\theta = (a, b, c, d)$. Recapping, our goal is to optimise these parameters so that the loss function is minimised. The method we are using right now is backpropagation which requires us to work out the gradients starting from the output backwards. So recalling that $\bar{\mathcal{L}} = 1$ is initialised we firstly differentiate the loss and residual:

$$\frac{\partial \mathcal{L}}{\partial r} = \hat{r} = r, \quad \frac{\partial r}{\partial \hat{y}} = \bar{\hat{y}} = 1$$

Differentiating the next layer (output operation $\hat{y} = ah + d$):

$$\bar{a} = \bar{\hat{y}}\frac{\partial \hat{y}}{\partial a} = \bar{\hat{y}}h = rh$$

$$\bar{h} = \bar{\hat{y}}\frac{\partial \hat{y}}{\partial h} = \bar{\hat{y}}a = ra$$

$$\bar{d} = \bar{\hat{y}}\frac{\partial \hat{y}}{\partial d} = \bar{\hat{y}} = r$$

Next differentiating the activation function $h = \tanh z$ yields:

$$\bar{z} = \bar{h}\frac{\partial h}{\partial z} = \bar{h}(1 - \tanh^2 z) = \bar{h}(1 - h^2) = ra(1 - h^2)$$

Finally, differentiating the first affine operation $z = bx + c$, yields:

$$\bar{b} = \bar{z}\frac{\partial z}{\partial b} = \bar{z}x = ra(1 - h^2)x$$

$$\bar{c} = \bar{z}\frac{\partial z}{\partial c} = \bar{z} = ra(1 - h^2)$$

Recalling that $\bar{u} = \frac{\partial \mathcal{L}}{\partial u}$ and that our parameters are $\theta = (a, b, c, d)$ we simply get the desired result:

$$\nabla_\theta \mathcal{L} = \begin{pmatrix} \bar{a} \\ \bar{b} \\ \bar{c} \\ \bar{d} \end{pmatrix} = \begin{pmatrix} rh \\ ra(1 - h^2)x \\ ra(1 - h^2) \\ r \end{pmatrix}$$

Remark 3.20. From the above, simple, example we can see a few variables being reused. This is incredibly valuable when our networks grow and something that we can exploit later.

Let $h \in \mathbb{R}^n, W \in \mathbb{R}^{m\times n}$ and $z, b \in \mathbb{R}^m$ with an affine layer given by $z = Wh + b$. Suppose that through backpropagation we have all the derivatives up to and including $\bar{z}$. We can write out the above layer in a componentwise form as:

$$z_i = \sum_{j=1}^{n} W_{ij}h_j + b_j$$

Differentiating with respect to h_j yields:

$$\frac{\partial z_i}{\partial h_j} = W_{ij}$$

And using the chain rule we get:

$$\frac{\partial \mathcal{L}}{\partial h_j} = \sum_{i=1}^{m} \frac{\partial \mathcal{L}}{\partial z_i}\frac{\partial z_i}{\partial h_j} = \sum_{i=1}^{m} \bar{z}_i W_{ij}$$

Or in a more convenient matrix form:

$$\bar{h} = W^T \bar{z}$$

This is the matrix form of the accumulation rule for the specific layer as the matrix W^T encodes all the information about the output neurons to which h_j contributes.

Suppose on the other hand, we now looked at the differential of the parameter W_{ij}:

$$\frac{\partial z_i}{\partial W_{ij}} = h_j \implies \frac{\partial \mathcal{L}}{\partial W_{ij}} = \bar{z}_i h_j$$

Or in the more conveninent matrix form:

$$\bar{W} = \nabla_W \mathcal{L} = \bar{z}h^T$$

In an analogous way we can easily yield:

$$\bar{b} = \nabla_b \mathcal{L} = \bar{z}$$

Let us now redefine h, so that it simply is governed by some activation function σ which acts componentwise. Namely:

$$h_i = \sigma(z_i)$$

Since each h_i depends only on z_i, if we perform backpropagation for z_i we have:

$$\frac{\partial \mathcal{L}}{\partial z_i} = \frac{\partial \mathcal{L}}{\partial h_i}\sigma'(z_i)$$

Or as an entire column vector:

$$\bar{z} = \bar{h} \odot \sigma'(z) \tag{3.4}$$

where $\odot$ is the Hadamard product.

Remark 3.21. The Hadamard product is an operation that acts to multiply two vectors componentwise. Namely for any $a, b \in \mathbb{R}^n$:

$$a \odot b = \begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{pmatrix} \odot \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix} = \begin{pmatrix} a_1 b_1 \\ a_2 b_2 \\ \vdots \\ a_n b_n \end{pmatrix}$$

Returning to the problem at hand if we now consider a network with multiple layers:

$$z_l = W_l h_{l-1} + b_l, \quad h_l = \sigma_l(z_l), \quad h_0 = x$$

Suppose σ_L is the identity, namely that the output activation function is the identity so that $h_L = z_L$. Choosing the usual loss function:

$$\mathcal{L} = \frac{1}{2}||h_L - y||^2$$

If we define δ_l as the gradient of the loss function with respect to z_l (or more commonly the adjoint), namely $\delta_l := \nabla_{z_l}\mathcal{L} = \bar{z}_l$, then it is not hard to see that at the output node we will get:

$$\delta_L = h_L - y$$

Now we can see the full power of backpropagation in deriving the subsequent adjoints. Given δ_{l+1} we can calculate the next layer δ_l. Consider the defining function for the next layer:

$$z_{l+1} = W_{l+1}h_l + b_{l+1}$$

Differentiating with respect to h_l yields:

$$\frac{\partial z_{l+1}}{\partial h_l} = W_{l+1}$$

Then consider:

$$\frac{\partial \mathcal{L}}{\partial h_l} = \frac{\partial \mathcal{L}}{\partial z_{l+1}}\frac{\partial z_{l+1}}{\partial h_l} = \delta_{l+1}W_{l+1}$$

by definition of δ. Switching operations under the transpose easily yields:

$$\bar{h}_l = W_{l+1}^T \delta_{l+1}$$

Then by definition of h_l and the activation function we have:

$$\frac{\partial \mathcal{L}}{\partial z_l} = \frac{\partial \mathcal{L}}{\partial h_l}\sigma_l'(z_l) = \bar{h}_l \odot \sigma_l'(z_l)$$

Giving the result:

$$\delta_l = \bar{z}_l = (W_{l+1}^T \delta_{l+1}) \odot \sigma_l'(z_l) \tag{3.5}$$

Remark 3.22. Whilst this may feel rather circular, the process actually falls out very nicely and precisely the inner mathematical structure of backpropagation. That is, calculating the gradients of the loss starting from the outputs and working backwards.

Doing the same thing but differentiating with respect to the weights W_l and the bias gradients b_l yields by an almost identical calculation:

$$\bar{W}_l = \nabla_{W_l}\mathcal{L} = \delta_l h_{l-1}^T \tag{3.6}$$

$$\bar{b}_l = \nabla_{b_l}\mathcal{L} = \delta_l \tag{3.7}$$

Equations 3.5, 3.6 and 3.7 are precisely the defining equations for backpropagation. Circling back to the start of the chapter, they are simply an iterative mathematical structure to determine gradients of multi-layered neural networks by utilising the local rules of each layer.

So why then do we use backpropagation? Well most simply, it tells us information about the gradient of the loss function with respect to *all* parameters used to train the model. Unlike

forward-mode AD which gives the derivative along one parameter direction, backpropagation gives sensitivities along all directions.

Importantly, backpropagation computes the full gradient at an efficiency on the same order as the original computation and avoids repeating it for every parameter. The tradeoff however is that it takes up a lot of memory, since the backward rules require the computer to store information about the preceding layers.

Remark 3.23. Whilst $\bar{W}_l$ and $\bar{b}_l$ are typically the quantities we care about when training, it is important to realise that the above equations clearly tell us that the gradients with respect to the input $\bar{z}_l$ play a key role in determining the end values. This means that the training inputs inherently influence how we train our model! Choosing different positions and therefore loss landscapes will dictate how we update our parameters!

Chapter 4

Small Projects: Linear Regression and a Multilayer Network

Part II

Practical Deep Learning

Chapter 5

Project: Function and Derivative Approximation

Part III

Physics-Informed Neural Networks

Chapter 6

Differential Equations as Learning Problems

6.1 Governing equations, residuals and loss functionals

Up until now, the pivotal part of training of a neural network has involved using observations (x_i, y_i) as a means of testing whether our model is 'accurate'. Namely, running the inputs through the model u_θ as $u_\theta(x_i)$, checking the comparison with the known values y_i via the loss and ultimately optimising this model so that $u_{\theta'}(x_i) \approx y_i$ for an 'optimised' choice of paramaters θ'.

Suppose instead that we knew a law that governed the unknown function. Take for instance:

$$u'(t) + u(t) = 0, \quad u(0) = 1 \tag{6.1}$$

Knowing this law gives us an additional basis for evaluating whether the models accuracy. Moreover, it also means that we don't need to know the correct values of a candidate function at every time, rather we can check whether it satisfies said law. For instance, let v be a candidate with a prediction $v(t)$ and rate of change $v'(t)$. In order to satisfy the above law, we would require that:

$$v'(t) = -v(t)$$

The disagreement provides a signal to us when training.

Remark 6.1. There are many differential equations we can use as a governing law. The above choice is purely arbitrary and will provide an easier introduction than a more general approach to learning PINN's.

Definition 6.2 (Forward initial-value problem)**.** A **forward initial value problem** (IVP) refers to the training of a neural network to predict the state of a system forward in time when the governing differential equations and initial and/or boundary conditions are known.

In our situation 6.1, the IVP is to find a $u \in \mathcal{C}^1([0, T])$ for some time $T > 0$ such that:

$$u'(t) + u(t) = 0 \quad (0 < t < T), \quad u(0) = 1$$

To do so, we approximate the solution by a neural network in the usual way:

$$u_\theta(t) = F(\theta, t), \quad \theta \in \mathbb{R}^p$$

As we train, we update our parameters θ and consequently choose a different function from the family $F(\theta, t)$ that is specified by the neural network architecture. Typically, when we build our networks we use smooth activation functions like tanh so that the candidate functions are smooth $u_\theta \in \mathcal{C}^1([0, T])$.

Definition 6.3 (Differential-equation residual)**.** A **differential-equation residual** is a quantitative measure of how much the neural network's approximate solution violates the governing physical law. It is the difference between the prediction of the network and the actual differential law. Let $\mathcal{N}[u](t) = f(t)$ be the governing equation of the system with $u(t)$ the solution. Then for a candidate solution v the residual is:

$$r_v(t) := \mathcal{N}[v](t) - f(t) \tag{6.2}$$

In our case, if we let $v \in \mathcal{C}^1([0,T])$ be a candidate for our case, then the residual $r_v(t)$ is:

$$r_v(t) = v'(t) + v(t) - (u'(t) + u(t))$$

But by the governing equation 6.1:

$$r_v(t) = v'(t) + v(t)$$

When we train our network, we pick a specific $u_\theta(t) = F(\theta, t)$ from the family of functions defined by our network architecture. In our defining law 6.1, the derivatives is with respect to time. As such, differentiating the neural network $F(\theta, t)$ with respect to t yields the residual as above:

$$r_\theta(t) := \frac{\partial F}{\partial t}(\theta, t) + F(\theta, t)$$

Remark 6.4. Note that $r_\theta(t)$ is simply the notation for the residual of the network with the specific parameters θ.

Using our intuition and as is clearly interpreted from 6.2, the residual measures the local violation of the neural network. Moreover, we clearly now have two important derivatives:

1. $\partial_t F(\theta, t)$ - constructs the residual
2. $\nabla_\theta \mathcal{L}(\theta)$ - determines a training update

AD can compute both. **Very importantly** though, because we now have derivative-dependent losses, we must also now differentiate computed input derivatives with respect to the parameters of our model. For instance, $\nabla_\theta \mathcal{L}(\partial_t F(\theta, t))$ which requires two passes through the computational graph. In the case that the law is defined by higher order differentials of varying variables, we may have many passes to count for all derivatives. These higher order derivatives are obviously calculated using the chain rule and can lead to very large expressions.

In an unsurprising way we can also account for the deviation from the initial condition by simply taking $r_i(t_0) := \sum_i v_i(t_0) - f_i(t_0)$ where v_i, f_i are the ith initial condition (i.e. for $i = 2, v'(t_0), f'(t_0)$ and each t_0 is the input for the initial condition. For instance in our case we simply have $v(0) - u(0) = v(0) - 1$. So now that we have a residual for the differential equation and for the initial conditions the next step is to be able to combine these to define the loss of the model. This is a relatively intuitive process, with the main care being an appropriate penalization for each term. This leads to the following definition:

Definition 6.5 (Continuum residual loss)**.** Let λ_r, λ_i be positive weights with $i \in \{0, ..., n-1\}$ where n is the order of the differential equation defining the IVP. Let r_i bet the ith initial condition of the IVP with input t_0. Then if $v \in \mathcal{C}^n([t_0, T])$ is a candidate solution, the continuum residual loss is the functional:

$$\mathcal{J}[v] = \lambda_r \frac{1}{T - t_0} \int_{t_0}^{T} |r_v(t)|^2 dt + \sum_{i=0}^{n-1} \lambda_i |r_i(t_0)|^2 \tag{6.3}$$

The first term penalises violations of the differential equation and susbequent terms, violations of the initial conditions. The $1/(T-t_0)$ makes the integral a mean squared residual over the interval. If we compose the functional with the neural-network representation $v(t) = u_\theta(t)$ then we get the parameter objective:

$$\theta \mapsto \mathcal{J}[u_\theta]$$

Now in our example, the continuum residual loss is much more digestible and simplifies to:

$$\mathcal{J}[v] = \lambda_r \frac{1}{T} \int_0^T |v'(t) + v(t)|^2 dt + \lambda_0 |v(0) - 1|^2 \tag{6.4}$$

Remark 6.6. In the case that the continuum loss is zero, this implies that the candidate function $u(\theta)$ is precisely the solution to the IVP. Or in other words it is the exact solution to the system governed by the law that we have modelled it with. Can we achieve this or will a network find it? Highly unlikely, but it gives us a quantifiable goal to work too... Minimisation of the loss.

Unfortunately, while Remark 6.6 points out the unlikeliness of achieving 0 continuum loss, we have a bigger problem at hand. When training a neural network we use a finite set of discrete observation tuples when training. This is a physical limitation of our computer systems. While we can increase the number of data points, we can't ever achieve continuity. As such, the idea of using a continuum residual loss is purely hypothetical. In order to train a PINN, we have to use collocation points and an empirical PINN loss:

Definition 6.7 (Collocation points)**. Collocation points** are the points $t_1, ..., t_N \in (0, T)$ at which we evaluate the residual of the model.

Definition 6.8 (Empirical Residual Loss)**.** Let $(t_1, ..., t_N) \in (0, T)$ be collocation points, $(\lambda_0, ..., \lambda_{n-1}, \lambda_r)$ be positive weights and $r_i(t_o)$ the ith initial condition residual for the initial input t_0. If $v \in \mathcal{C}^n([t_0, T])$ is a candidate solution, then the **empirical residual loss** is the functional:

$$\mathcal{J}_N[v] = \frac{\lambda_r}{N} \sum_{j=1}^{N} |r_v(t_j)|^2 + \sum_{i=0}^{n-1} \lambda_i |r_i(t_0)|^2 \tag{6.5}$$

Then we can easily define the empirical PINN loss as the composition with the parameter objective:

Definition 6.9 (Empricial PINN Loss)**.** Let $\mathcal{J}_N[v]$ be an empirical residual loss and $u_\theta(t) = v(t)$ be the parameter objective associatied with the candidate solution v. Then the **empirical PINN loss** is defined as:

$$\mathcal{L}_N(\theta) := \mathcal{J}_N[u_\theta] \tag{6.6}$$

In our case, the empirical PINN loss becomes:

$$\mathcal{L}_N(\theta) = \lambda_r \frac{1}{N} \sum_{i=1}^{N} |\partial_t F(\theta, t_i)|^2 + \lambda_0 |F(\theta, 0) - 1|^2$$

An important point to note is that a small training loss and a small solution error represent different quantities. Understanding them requires analysis the differential equation and the sampling.

6.2 Residual-Error Relations and Stability Estimates

Whilst we know about the loss of a PINN, how do we evaluate the error of the network? To do so generally, we will set up the following IVP. Consider an arbitrary n^{th} order IVP:

$$u^{(n)}(t) = g(t, u(t), u'(t), ..., u^{(n-1)}(t)), \quad t \in [t_0, T]$$

with initial conditions:

$$u^{(i-1)}(t_0) = a_i, \quad i = 1, ..., n$$

Assuming a solution $u \in \mathcal{C}^n([t_0, T])$ exists, then for a candidate solution $v \in \mathcal{C}^n([t_0, T])$ we can write the differential-residual as:

$$r_v(t) := v^{(n)}(t) - g(t, v(t), v'(t), ..., v^{(n-1)}(t))$$

and the residual initial conditions as:

$$\delta_i := v^{(i-1)}(t_0) - a_i$$

For sake of convenience, we introduce the following vectors:

$$U = \begin{pmatrix} u \\ u' \\ \vdots \\ u^{(n-1)} \end{pmatrix}, \quad V = \begin{pmatrix} v \\ v' \\ \vdots \\ v^{(n-1)} \end{pmatrix}, \quad \delta = \begin{pmatrix} \delta_1 \\ \vdots \\ \delta_n \end{pmatrix}$$

and define:

$$F(t, x) = \begin{pmatrix} x_2 \\ \vdots x_n \\ g(t, x_1, ..., x_n) \end{pmatrix}, \quad b = \begin{pmatrix} 0 \\ \vdots \\ 0 \\ 1 \end{pmatrix}$$

When $n = 1$, then $F(t, x) = g(t, x)$ and $b = 1$. The governing equation and candidate equation can thus be written as:

$$U' = F(t, U), \quad V' = F(t, V) + br_v$$

Defining the state error as

$$E(t) := V(t) - U(t)$$

we then get the following:

Proposition 6.10. *For the IVP as above the* ***general residual-error equation*** *is given by:*

$$E(t) = \delta + \int_{t_0}^{t} [F(s, U(s) + E(s)) - F(s, U(s)) + br_v(s)]ds \tag{6.7}$$

Proof. Differentiating the state error yields $E'(t) = V'(t) - U'(t)$. Rearranging $V = U + E$ and integrating yields the result. □

For our example we have $u(t) = e^{-t}$ so that 6.7 simplifies as:

$$e(t) = e^{-t}\delta + \int_{t_0}^{t} e^{-(t-s)} r_v(s)ds$$

Remark 6.11. This precisely shows us the differing sources of error. We have the error $e^{-t}\delta$ which is the error inherited from the initial condition and the integral term which reflects the error accumulated from violations of the differential equation.

From this equation, it is possible to find an error bound, but to do so we need to know about the dynamics of the system. In the case that we have a fixed equation, interval, positive weights and can guarantee a stability assumption about the system, then we can ensure that $\mathcal{J}[v] \to 0$. If this holds then we have uniform convergence of the state error to 0. To get a more intuitive understanding, we will turn back to our example to control the solution error in the continuum loss case.

Under a relatively easy application of the triangle inequality, the Cauchy-Schwarz inequality and then evaluating the integral, one yields a bound on the residual-error:

$$|e(t)| \leq e^{-t}|\delta| + \sqrt{\frac{1-e^{-2t}}{2}}||r_v||_{L^2(0,t)}$$

Moreover, we can further this inequality to gain a bound on the residual of our candidate solution. A very simple approximation of the bounds of $|\delta|$ and $||r_v||_{L^2(0,T)}$ from 6.4 yields the following uniform estimate:

$$||v-u||_{L^\infty(0,T)} \leq \left(\frac{1}{\sqrt{\lambda_0}} + \sqrt{\frac{T(1-e^{-2T}}{2\lambda_r}}\right)\sqrt{\mathcal{J}[v]}$$

Remark 6.12. Note that this bound is specific to our example. It does not guarantee the control of the uniform solution error in general. Indeed it further relies on the continuum loss and in reality the loss of our networks are represented by the empirical loss. To get a similar statement we require not only the stability of the differential equation (so that $\mathcal{J}[v] \to 0$ but also an argument regarding how we control the residual between collocation points.

6.3 From empirical loss to solution accuracy

As has become clear from the proceeding section, the stability of the continiuum residual gives valuable information about analytic functions. However, in a training a PINN this basically unachievable as we don't have a continuum of collocation points. As such in training our goal is to minimise the empirical loss.

The previous control of the solution error with the continuum loss also fails since we can have zero empirical loss whilst having arbitrarily large solution error.

Proposition 6.13 (Finite collocation does not determine a smooth solution)**.** *Consider the nth-order scalar IVP:*

$$u^{(n)} = g(t, u', ..., u^{(n-1)}), \quad u^{(i-1)}(t_0) = a_i, \;\; i = 1, ..., n$$

and suppose $u \in \mathcal{C}^n([t,T])$ is an exact solution. Let $t_1, ..., t_N \in (t_0, T)$ be finitely many fixed, distinct collocation points. Then there exists candidate functions $v_A \in \mathcal{C}^n([t_0, T])$, indexed by $A \in \mathbb{R}$, such that:

$$\mathcal{J}_N[v_A] = 0$$

for every A whilst the solution error grows indefinitely:

$$||v_A - u||_{L^\infty(t_0,T)} \to \infty, \quad \text{as } |A| \to \infty$$

Chapter 7

Project: An ODE PINN

7.1 Goals

7.2 Theory and Derivations

7.3 Exercises and Implementation

7.4 Results and Reflection

7.5 References

Chapter 8

Project: The Heat Equation

8.1 Goals

8.2 Theory and Derivations

8.3 Exercises and Implementation

8.4 Results and Reflection

8.5 References

Chapter 9

Inverse Problems

9.1 Goals

9.2 Theory and Derivations

9.3 Exercises and Implementation

9.4 Results and Reflection

9.5 References